

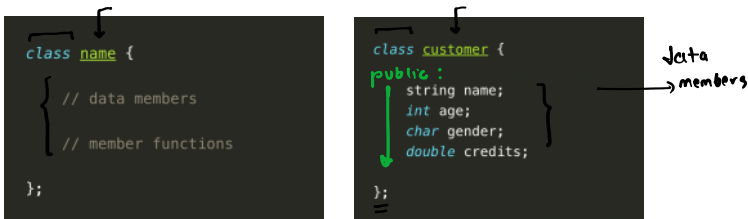
'M' or 'F'

Object Oriented Programming

It is a **style of programming** that allows you to very conveniently **model** real-world **entities**.

<code>vec<string></code>	name									...
<code>vec<int></code>	age									...
<code>vec<char></code>	gender									...
<code>vec<double></code>	credits									...

	0	1	...	i	
<code>vec<cust></code>	c1	c2	c3	c4	c5
	name	name	name	name	name
	age	age	age	age	age
	gender	gender	gender	gender	gender
	credits	credits	credits	credits	credits
					...



- A **class** is a **blueprint** of the entity you are trying to model e.g. customer.
- A **class** in itself does not consume any memory.
- An **object** is an **instance** of the class and consumes memory.

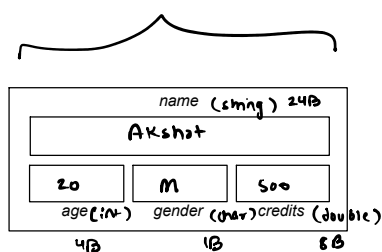
`int x;`
`char ch;`
`string s;`

4B
x

1B
ch

s

(obj)
 var: name
 customer c1;
 type
 (class)

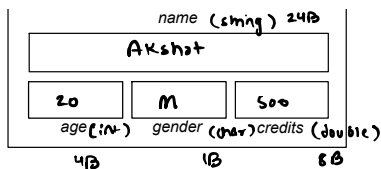


`c1.name = "Akshat"`
 instance member

```
customer c1;
```

type
(class)

```
c1.name = "Akshat"
c1.age = 20
c1.gender = 'M'
c1.credits = 500;
```



objname = member

➤ Constructor

It is a **special member function** of a class used to **initialize** a class object.

- it has the **same name as the class name**
- it **does not have a return type**
- it is **invoked automatically** upon declaration of an object to **construct** it.

```
customer c1;
```

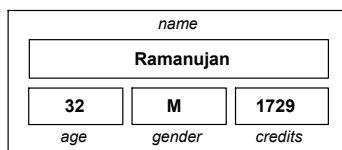
when you create c1,
 then default const
 is invoked to
 create c1

Zero
 param
 const

A **parameterized** constructor is a class constructor to which we pass **arguments** corresponding to initial values of data-members of the object being constructed.

```
customer(string n, int a, char g, double c) {
    name = n;
    age = a;
    gender = g;
    credits = c;
}
```

```
customer c("Ramanujan", 32, 'M', 1729);
```



Once you provide an implementation of a parameterized constructor, the compiler **no longer** provides an implicit default constructor.

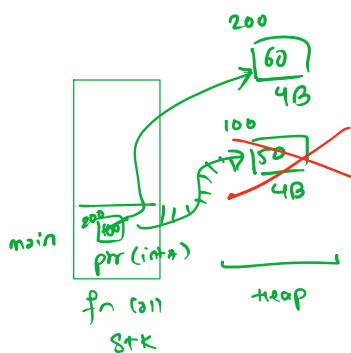
➤ Destructor

It is a **special member function** of a class used to **destruct** a class object.

- it has the **same name as the class name** (preceded by ~)
- it **does not have a return type**
- it **does not accept any arguments**
- it is **invoked automatically** once an **object goes out of scope** to **destruct** it.

Unlike a constructor, a destructor cannot be **overloaded**.

```
main() {
    int* ptr = new int;
    *ptr = 50;
    cout << *ptr; //50
    delete ptr;
    ptr = new int;
    *ptr = 60;
    cout << *ptr; //60
}
```





Operator Overloading

```
int x = 10;
int y = 20;

x > y ? cout << "x > y" : cout << "x < y" ;

string s = "hello";
string t = "world";

cout << s + t << endl;
```

```
int x = 10;
int y = 20;
cout << (x + y); // 30

string s1 = "abc";
string s2 = "def";
cout << (s1 + s2); // "abc def"

customer c1 ("Ramanujan", 32, 'm', 1729);
customer c2 ("Aryabhata", 74, 'm', 0);
cout << (c1 + c2);
```

Operators are overloaded by means of **operator functions**, which are regular functions with **special names** — their name begins by the **operator** keyword followed by the **operator sign** that is overloaded.

```
inline operator sign( parameters ) { /... body .../ }
```

Overloadable operators

>	=	-	*	/	%
>>	+=	--	++	./	

>>

= >= <= !=
= > < >= <= % * ^ ! |
- & ~ = && || %> [] 0 , -> new
delete new[] delete[]

$\text{frame} (\text{---}) \{$
 $\dots$
 $\}$
 $\text{operator} + (\text{---}) \{$
 $\vdots$
 $\}$

`<customer c1 ("Ramanujan", 32, 'm', 1729);`
`<customer c2 ("Aryabhata", 74, 'm', 0);`

`c1 + c2 ;`

↓

`c1.operator + (c2)`

$\Rightarrow () \rightarrow$ fn call operator

class CMP {

main() {
 cmp obj;
}

function obj. / functors



```
graph LR; main["main()"] --> functors["function obj. / functors"]
```

⇒ () → ...

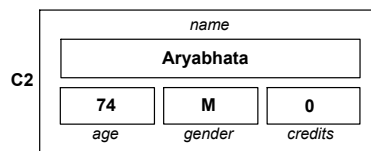
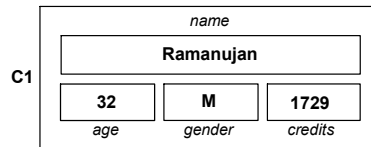
```
class CMP {
public:
    .... operator()(.....) {
        ....
    }
    .... operator + (.....) {
        ....
    }
};
```

```
main() {
    CMP obj;
    obj.operator()(.....); or obj(.....);
    CMP obj2;
    cout << obj + obj2;
    obj.operator + (obj2);
}
```

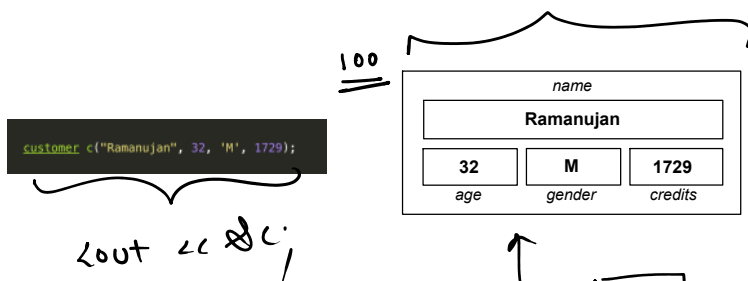
```
bool operator>(const customer& c) {
    return age > c.age;
}
```

```
customer c1("Ramanujan", 32, 'M', 1729);
customer c2("Aryabhata", 74, 'M', 0);
c1 > c2 ? cout << "c1 > c2" : cout << "c1 < c2";
```

c1.operator > (c2)



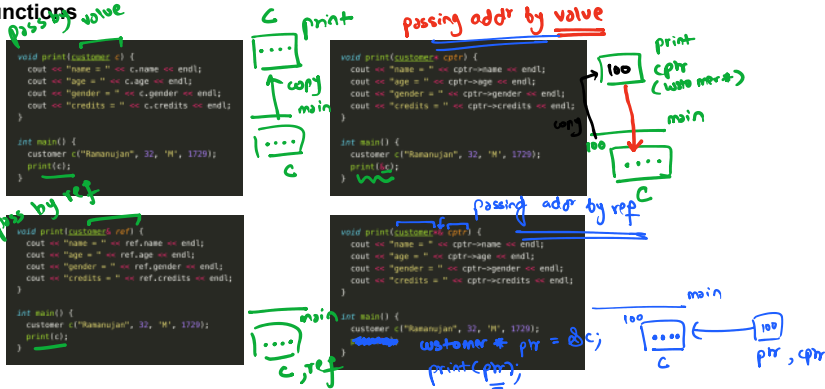
Pointers and Objects



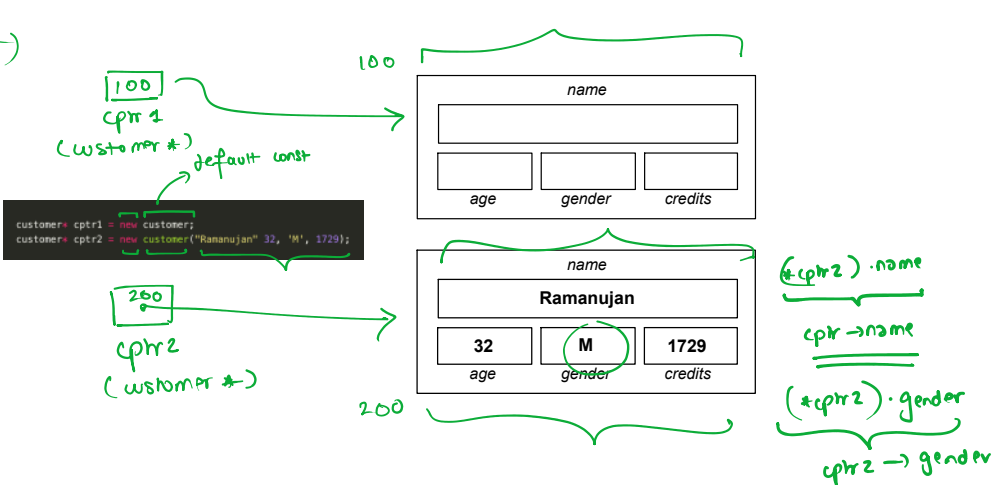
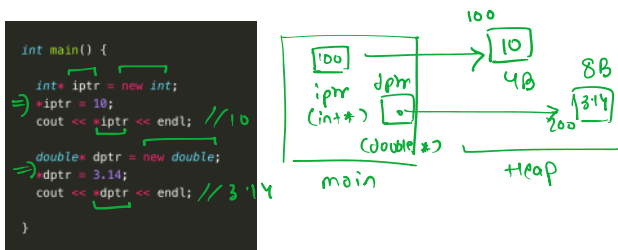
customer * cptr = &c;

(*cptr) . name / cptr → name (customer*)
 (*cptr) . age / cptr → age
 (*cptr) . gender / cptr → gender
 (*cptr) . credits / cptr → credits

Passing Objects to Functions

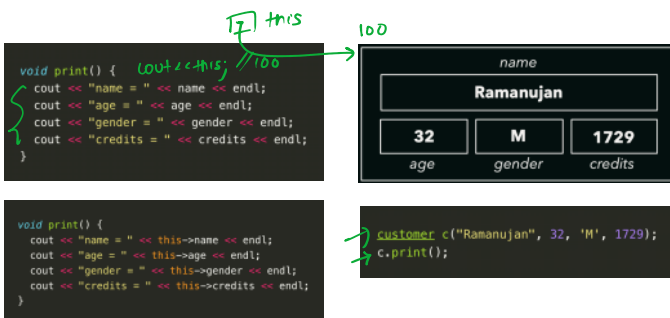


Creating Objects on Heap Memory

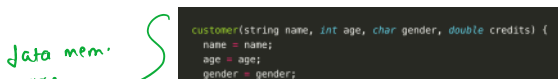


This Pointer

this is a **const pointer** to an object on which a **member function** is invoked.



The **this** pointer can be used to disambiguate a parameter with the same name as a data member within a member function.



Data mem.
are
shadowed
by params

```
customer(string name, int age, char gender, double credits) {  
    name = name;  
    age = age;  
    gender = gender;  
    credits = credits;  
}
```

```
customer(string name, int age, char gender, double credits) {  
    this->name = name;  
    this->age = age;  
    this->gender = gender;  
    this->credits = credits;  
}
```